RESUMEN MEDIANTE EL METODO CORNELL

Capitulo 1: Valores, Tipos y Operadores

Estudiante	Emmanuel Josue Rozotto Diaz
Carne	2890 20 23374
Libro	Eloquent JavaScript
Curso	Desarrollo Web

Objetivo del trabajo

Organizar las ideas principales del capitulo con el metodo Cornell, explicandolas con palabras propias y usando ejemplos cortos de JavaScript para comprobar como funcionan.

Estructura Cornell

La columna izquierda contiene preguntas o palabras clave para repasar. La columna derecha desarrolla los apuntes y ejemplos. Al final se incluye una sintesis general del aprendizaje.

Apuntes Cornell

CONCEPTOS Y PREGUNTAS	APUNTES Y EJEMPLOS

¿Que son los valores y los tipos?	Un valor es una pieza de informacion que JavaScript puede guardar y utilizar. Cada valor tiene un tipo, y ese tipo indica que operaciones tienen sentido. Por ejemplo, un numero sirve para calcular, mientras que una cadena sirve para representar texto.
	`console.log(typeof 42);  // "number"`
	`console.log(typeof "hola");  // "string"`

Numeros y precision numerica	JavaScript representa los numeros con una cantidad limitada de bits. Los enteros pequeños suelen ser exactos, pero algunos decimales solo pueden almacenarse como aproximaciones. Por eso, en calculos de dinero conviene redondear y no confiar en una igualdad decimal exacta. `console.log(0.1 + 0.2); // 0.30000000000000004`

Aritmetica y precedencia	Los operadores +, -, *, / y % permiten hacer calculos. La multiplicacion, la division y el residuo se evaluan antes que la suma y la resta. Los parentesis ayudan a dejar claro el orden que se desea.
	`console.log(100 + 4 * 11);   // 144`
	`console.log((100 + 4) * 11); // 1144`

Residuo o modulo (%)	El operador % devuelve lo que sobra de una division. Es util para saber si un numero es par o impar, o para trabajar con ciclos repetitivos. `console.log(17 % 2); // 1, por eso 17 es impar`

Numeros especiales	Infinity y -Infinity representan resultados que van mas alla de los numeros normales. NaN significa que una operacion numerica no produjo un resultado valido. Aunque NaN pertenece al tipo number, no representa una cantidad util.
	`console.log(0 / 0);  // NaN`
	`console.log(typeof NaN);  // "number"`

Cadenas y template literals	Las cadenas representan texto y se escriben con comillas simples, dobles o acentos graves. Los template literals permiten insertar expresiones con ${...} y tambien escribir texto en varias lineas.
	`let nombre = "Emmanuel";`
	`console.log(`Hola, ${nombre}. El doble de 6 es ${6 * 2}.`);`

Concatenacion y caracteres especiales	El operador + une cadenas. La barra invertida permite escribir caracteres especiales, como \n para un salto de linea o \t para una tabulacion.
	`console.log("Desarrollo" + " " + "Web");`
	`console.log("Linea 1\nLinea 2");`

CONCEPTOS Y PREGUNTAS	APUNTES Y EJEMPLOS

Valores booleanos y comparaciones	Los booleanos solo pueden ser true o false. Los operadores >, <, >=, <=, ==, !=, === y !== producen resultados booleanos y se usan para tomar decisiones en un programa.
	`let edad = 18;`
	`console.log(edad >= 18); // true`

Operadores logicos	El operador && exige que ambos lados sean verdaderos; || acepta que al menos uno sea verdadero; y ! invierte un valor booleano. Sirven para formar condiciones mas completas.
	`let tieneCuenta = true;`
	`let claveCorrecta = false;`
	`console.log(tieneCuenta && claveCorrecta); // false`

null y undefined	Los dos expresan ausencia de informacion, pero no se usan exactamente igual. null suele colocarse de forma intencional para indicar que no hay un valor. undefined aparece cuando falta una asignacion o una propiedad no existe.
	`let seleccion = null;`
	`let pendiente;`
	`console.log(seleccion, pendiente); // null undefined`

Conversion automatica de tipos	JavaScript a veces convierte un tipo en otro para completar una operacion. Esto puede ser comodo, pero tambien confuso: con + puede unir texto, mientras que con * intenta convertir el texto a numero.
	`console.log("5" + 2); // "52"`
	`console.log("5" * 2); // 10`

Diferencia entre == y ===	== compara despues de permitir conversiones automaticas. === compara el valor y tambien el tipo, por lo que evita resultados inesperados. Como regla general, es mejor usar === y !==.
	`console.log(5 == "5");  // true`
	`console.log(5 === "5"); // false`

Cortocircuito: &&, || y ??	Estos operadores pueden devolver uno de sus operandos sin evaluar todo. && se detiene con el primer valor falso; || toma el primer valor verdadero; y ?? usa la alternativa solo si el valor anterior es null o undefined.
	`let usuario = null;`
	`console.log(usuario ?? "Invitado"); // "Invitado"`
	`console.log(0 || 10);  // 10`
	`console.log(0 ?? 10);  // 0`

Resumen final

1. El capitulo explica que los programas trabajan con valores, y que cada valor pertenece a un tipo que determina como puede utilizarse.

2. Los numeros permiten realizar operaciones aritmeticas, aunque algunos decimales se guardan como aproximaciones y pueden mostrar pequenas diferencias de precision.

3. Las cadenas representan texto y pueden construirse con comillas o con template literals para insertar expresiones de una manera mas clara.

4. Los valores booleanos, true y false, aparecen cuando se comparan datos y son esenciales para expresar condiciones.

5. Los operadores aritmeticos transforman numeros, mientras que los operadores de comparacion y logicos ayudan a evaluar situaciones.

6. Los valores null y undefined representan ausencia de informacion, pero null normalmente es intencional y undefined suele indicar que algo todavia no tiene valor.

7. La conversion automatica de tipos puede facilitar algunas expresiones, aunque tambien produce resultados inesperados si no se entiende lo que JavaScript esta haciendo.

8. Por esa razon, considero mas seguro usar === en lugar de == cuando necesito una comparacion estricta.

9. Los operadores &&, || y ?? aplican cortocircuito y permiten escoger valores o evitar evaluaciones innecesarias.

10. En conjunto, los valores, los tipos y los operadores forman la base para procesar informacion, construir condiciones y escribir programas mas claros.

Conclusion personal

Lo que mas me ayudo de este capitulo fue entender que JavaScript no siempre trata los datos como uno espera. Dos valores pueden parecer iguales, pero tener tipos diferentes, y una operacion puede provocar una conversion automatica. Conocer estas reglas desde el principio ayuda a evitar errores y a leer mejor el resultado del codigo.

Fuente consultada

Haverbeke, Marijn. Eloquent JavaScript, capitulo 1: "Valores, Tipos y Operadores".
https://www.eloquentjavascript.es/01_values.html